H · Prove 全流程算子级清单与数据流（FPGA 实现骨架）

> **定位**：把一次 zk prove 从**入口到最底层叶片**涉及到的**每一个算子**，细化到**简单算子级**（域加/域乘/求逆、i64→Fr、数据切分/拼接/重排/组装、MSM、sumcheck 轮内折叠、EQ、折叠、挑战采样等），并让**数据流全程贯穿、无遗漏**。读者是 FPGA 实现者：能照着把每个算子落成一个硬件微操作/流水级。 **主线跟着 G2 的入口函数逐段展开**：`Prover::prove → chunked_prove_local → (Split->ChunkProver->Final) → prove_chunk → (witness 批量承诺 + 逆拓扑逐节点 sumcheck) → commit_prover.prove(合并 sumcheck + RLC) → batch_commit(MSM) → HyperKZG::open(折叠 + 3点批量打开)`。 **本章目标**：G 只给了 12 个粗粒度算子（G §5.5）；本章把这些算子**再往下拆成可直接综合的微算子**，并把它们之间的**数据流（谁产出谁消费、什么形状、怎么搬）串成一条无断点的链**。

符号约定：`F=Fr=ark_bn254::Fr`（BN254 标量域，模 `p = 0x30644e72e131a029b85045b68181585d97816a916871ca8d3c208c16d87cfd47`，254bit）；`Element=i64`(8B)；MLE=多线性扩展（超立方体表的求值形式，`2^k` 元素数组）；`Trace`= `HashMap<NodeId, Step>`。域基元（add/sub/mul/inv）在 Bn254Fr 上实现，MSM 在 BN254 曲线 G1 上。

> **章节索引**：**主链 段0~段7** 严格按 G2 主链排序（入口→逐步下钻到打开），是**一次 prove 的数据流主线**（每段产出下一段的输入）；**附录 A/B/C/D** 是按**硬件单元/算子类别**的查阅表（A=数据切分/拼接/组装，B1~B4=Lookup/embedding/卷积掩码等查表层，C=域求逆/除法，D=底层密码单元：域算术/EQ/MSM/transcript），供 FPGA 按单元查。各模型的 MSM/sumcheck **调用次数**见 F §7。**先读主链段0→段7 立起数据流，再按需查附录 A/B/C/D 落硬件**。

---

0. 全算子总览（先给地图，再逐段拆）

一次 prove 的算子是**「数据载体 → 算子(做什么变换) → 数据载体」**的链：上游算子的输出恰是下游算子的输入。下面这张竖直管线图是全文的时序骨架——每一行读作「这一步喂进什么数据、过哪个算子、得到什么」。**域标**：`i64`=8B 域化前；`F`=Bn254Fr；`G1`=曲线点；`∇(MLE)`=`DensePolynomial<Fr>` 求值表，`2^n` 元素数组。

```
① 入口
   Trace<Element=i64>                          ── Split / 替换被切节点（数据搬运·子集选取）
   | chunk_trace + 执行图(0=Split,C×ChunkProver,C+1=Final)   ▫ 只切表、不碰数值
   ▼  ──────── 每个 chunk 进入 prove_chunk ────────
② 域化与算子化 (段1)
   Element=i64 张量[···]  ──(a) to_field() i64→F──▶ Tensor<Fr>      ▫ 域转化 8B→32B,形状不变
   | ──(b) pad 到 2^n 幂──────────▶ Tensor<Fr> 尾部补0              ▫ 补0,记录 unpadded_shape
   | ──(c) into_mle()──────────────▶ ∇ MLE 求值表                   ▫ 行主序重解释,样本域=F
   ▼
③ witness 批量承诺 (段2)
   逐节点 ∇〔输入/中间输出〕MLE
   | ──切块(每2^20) + batch_poly_msm  Pippenger──▶ 每块 1 个 G1 承诺   ▫ 承诺=MSM C=Σaᵢ(g^τ^i)
   ▼
④ claims 生成 (段3)
   各节点输出 ∇ ── evals(point)+EQ ──▶ Claim{ point:F^n , value:F }    ▫ 断言"MLE 在点上的求值"
   ▼
⑤ 节点级 sumcheck (段4)
   该节点 虚拟多项式(多∇乘积×Challenge)
   | ── 逐轮 fix_low(fold 减半) + build_uni_poly ──▶ 轮多项式 fᵢ(x)   ▫ 把"断言"交给折叠值收口
   |                    (每节点各一轮;查表层走附录B1 LogUp GKR+链接sumcheck)
   ▼
⑥ 合并 sumcheck + RLC (段5)
   全 Claims ── EQ + batching ──▶ 归约到单点值
   | ── linear_combination(RLC=Σcᵢ Pᵢ) ──▶ 1 条 rlc_poly (∇)
   ▼
⑦ KZG 打开 (段6)
   rlc_poly + 固定点 ──(i) 折叠 2^m→常数 ──▶ 折叠多项式们
   | (ii) 批量承诺折叠poly + 采挑战 r ──▶ ℓ-1 个 G1
   | (iii) 3点 u=[r,-r,r²] batch open(MSM) ──▶ OpeningProof(W₀,W₁,W₂)
   ▼
⑧ 序列化 (段7)
   OpeningProof + 各 ChunkProof ── postcard + CanonicalSerialize ──▶ Proof 字节流 ──▶ verifier
```

> 读图要点：**数据自始至终只有一种"数值流通证"变形**——`i64 → F`(②a) 一次域化后，余下全程在 `F` 与 `G1` 上流转（样本 / 承诺 / 断言 / 轮多项式）。变量被**折叠**吸收（④⑤⑥⑦的 fold），不随 s 增涨；瓶颈点（MSM、sumcheck、fold 轮）就是「关键技术事实」列的三个重计算算子。逐项展开见 段0~段7；每条算子的 FPGA 硬件单元查 附录 A/B/C/D。

> **①~⑧ 每步的执行位置（细到算子，图读数到算子行）**——"这一步喂什么、过什么、在 FPGA 里放哪"逐行对账：

步	算子（做什么）	需要的数据输入	怎么来：部自足 / 预置 / 需主机喂
①	`split_in_chunks`→`build_chunks`→`replace_splitted_nodes`→`build_execution_graph`：切块+调度（搬运）	`full_trace` + `Model<Element>`（权重）	**主机喂**开（只喂一表）
①b	`to_verifier_io`：公共 IO i64→F（域化）	trace 边界	内部
②	`to_field`(i64→Fr) `pad_to_2pow` `into_mle`(→ ∇)	逐中间量	内部（纯据变形）
③	`gen_lookup_witness`(查表计数→multiplicity) `batch_commit`=`batch_poly_msm`（Pippenger MSM，两两 2^20 切块，≤128）	witness ∇们；**MSM 基点 = SRS** `g1_powers`	切块/切块略=代码常(2^20/128内部；**MS基点需 S预置**；模表承诺已setup 现**预置**
④	`evals(point)`+EQ 构造+挑战采样 → `Claim{点,值}`	节点输出 ∇；model_ctx	**挑战=内transcrip**model_ct预置
⑤	逐节点 `op.prove`：`fix_axes`→虚拟多项式→逐轮 `fix_low`(fold)+`build_uni_poly` sumcheck；查表层走 LogUp GKR+链接 sumcheck（附录 B1）	节点 ∇+claims	内部；**每挑战=内transcrip**
⑥	`claims_batching_sumcheck`(EQ+batching)→单点；`linear_combination`(RLC=Σcᵢ·Pᵢ)→1 条 rlc_poly	全部 claims；EQ 的 model/表**承诺**(预置)	内部；挑内建；**依预置承诺与 EQ**
⑦	`HyperKZG::open`：逐变量折叠→承诺折叠 poly→3点 `u=[r,-r,r²]` `kzg_open_batch`(MSM)	rlc_poly+单点；**SRS** `g1_powers`	折叠/3点地内部；**诺与打开MSM 需SRS 预置**挑战 r,q 建
⑧	`CanonicalSerialize` + postcard：Fr→32B、G1→2×32B 打包	各段 Proof 字段	内部（纯地）；**Pro字节交给机**

★ FPGA 端只实现①~⑧，能否完整跑通一次 prove？——能，且正好闭环（核实自 `zkml/src/iop/prover.rs:995/542/837`、`zkml/src/poly_commit/context.rs:149-210`、dp-crypto `arkyper/mod.rs:329-383`）。三种输入各有归属：

- **完全内部自足（FPGA 板上算子即可）**：i64→Fr / pad / into_mle（②）、MSM 切块策略(2^20、≤128 是代码常量)、witness 生成(算自 trace)、逐节点 sumcheck、RLC、HyperKZG 折叠、kzg_open_batch、序列化——**以及全链 ④⑤⑥⑦ 的 Fiat-Shamir Blake3 挑战采样**（这是最容易被忽略的"内建件"：只要挑战由主机出，①~⑧ 就不自足、会逐轮爆炸）。
- **必须 setup 预置（一次性，随 s/多项式大小生成，FPGA 提前装载）**：`SRS/g1_powers`（尺寸 `1<<max_num_vars`，**与序列长 `s` 绑定**，`context.rs:166`）、`model_commitments`（含表承诺，`context.rs:210`）。**③⑦ 的 MSM 基点和 ⑥ 的 EQ 都靠它们**；不预置则无法闭环。两者用 `CanonicalSerialize` 存到 FPGA 内存（换形状/模型需重装）。
- **主机只在首末交互**：开头喂 `full_trace + 权重`，末尾收 `Proof` 字节（⑧）。

两点确认过的边界：

- **求逆在主链非必需**（各层 provable 的 shift 用乘 `lt_eval` 处理，`generic_prove.rs:244-254`，无 `inv()`）；仅 BatchFFT 旁路用两处 inverse（`prover.rs:345/474`）。若不做 conv/FFT-proof 层，⑦ KZG / ⑥ RLC / ⑤ sumcheck 全程无求逆——FPGA 可省掉求逆单元（附录 C 只留作 FFT 旁路备件）。
- 线程并行不在此图内：① 的多个 ChunkProver 与 ⑤ 的多个节点可块间/节点间并行（见段0「多 chunk → Final」），FPGA 端即块数=并行度上限。

核心事实（对联 G2 §1.2/§1.3/[F]）：**证明的输入是「执行图 + 整份 Trace 表」**，不是逐 token。trace 是前向推理把每个图节点的中间量（i64 张量）记下来的表；prove 全程按 `backward_iter` 逆拓扑**消费**这张表，把 i64 量域化、承诺、做 sumcheck、最后归约到一个打开。**没有"逐 token"状态，只有"图序 + 表"** ——这是 FPGA 端到端内核能成立的结构前提。

段 0 · 入口与调度：trace 切块、执行图、切 chunk

入口链（G2 §2）：`Driver::prove`(model/llm.rs:774) → `Prover::chunked_prove_local`(iop/prover.rs:995) → `build_local_execution_graph` + `run_execution_graph`。这层**只做数据切分、组装、调度，不做任何域运算**。

微算子	源码位置	输入 → 输出	FPGA 判定
选集块策略	`iop/prover.rs:1019` `split_in_chunks`	trace 尺寸 → chunking_strategy	主机决策（逻辑，非计算）
切模型图成 chunk	`chunking.rs:267` `ModelChunk::build_chunks`	模型图 →`Vec<ModelChunk>`(子图+边界边+replaced_nodes)	图数据结构操作
trace 原地标记被切节点	`iop/prover.rs:1021` `replace_splitted_nodes`	track + split_info → 就地改	数据结构
取公共输入	`iop/prover.rs:1023` `to_verifier_io`	trace ii/o → 域化 IO	进入段1
建执行图	`iop/prover.rs:891/918-936`	chunks → ProverGraph(0=Split,1..C=ChunkProver,C+1=Final)	调度 DAG，非计算
切 chunk trace	`chunking.rs:553` `chunk_trace`	full_trace → 只取子图内节点 Step 的 Trace	**数据搬运/子集选取**
取 chunk 层引用	`chunking.rs:579` `chunk_layers`	model + chunk → 该 chunk 的层引用	索引
跨 chunk 挑战连续性	`prover_graph.rs:157-224`	共享 challenge → 逐 chunk 初始化 transcript	Fiat-Shamir（段6基元）

段0 数据流：`full_trace(steps: HashMap<NodeId,Step>)` 是唯一入口。`Split` 节点把它按 chunk 切出若干 `chunk_trace`（每个只含本块节点 Step），`ChunkProver(i)` 消费 `chunk_trace+chunk_layers+challenge`。**数据形状**：每块仍是 `HashMap<NodeId, Step>`，Step 里是 `Vec<TensorHandle>`（i64 张量句柄）。**无域运算**。

多 chunk → Final 合并（补齐主链最后一环）：`build_execution_graph`/`LocalProverCtx` 造出 `C+2` 个执行图节点（G2 §2.5）——`0=ProverSplit`、`1..=C` 各一个 `ChunkProver(i)`、`C+1=Final`：

```
ProverSplit ──chunk_trace_i──▶ ChunkProver(1..C)  每块各自 prove_chunk(段1~段6) 产出一个 ChunkProof
      各 ChunkProof ──────────────▶ Final  合并成单个完整 Proof
```

- `ProverSplit.run`（`prover_graph.rs:157-224`）：按 C 切 `chunk_trace`、初始化各块 transcript + `challenge_scalar`、做 `dry_run`（首跑预算）。**每块自此是"同一次证明的独立并行支路"**——这是 FPGA 在块间铺并行的地方。
- `ChunkProver(i).run` 调 `prover.prove_chunk(...)`，即本主链段1~段6 的一遍，产出 `ChunkProof`。
- `Final.run` 把 C 个 `ChunkProof` **合并**（`run_execution_graph` / 聚合 L1_claim 与各打开）成**一个完整 Proof** 输出（prover_graph.rs Final 段）。

默认 `num_chunks=None/Some(1)` 时只有 1 块，ProverSplit 不切、Final 直接收；`C>1`（水平切分或显式多块）才真正走"多块各自证、Final 合并"这条**并行聚合链**。**对 FPGA**：块数 C 就是端到端内核的并行度上限；单块情形整条链退化成纯串行，聚合节点几乎零额外算子。

段 1 · 域化与算子化：i64 → Fr → MLE

> **入口链**（G2 §3.3 主循环 `:671`）：`chunk_trace.get_step(node_id)` 取节点 Step → `output_tensors().to_field().pad_next_power_of_two()` → `into_mle()`。这是**逐中间张量的固定三步**，也是 FPGA 里最贴 trace 的一次大规模搬运+转换。

1.1 i64 → Fr（`to_field`）

微算子 源码位置 输入 → 输出 说明 **i64→Fr** `quantization/mod.rs:263-267` `ToField for Element`；`ark_ff` `F::from(i64)` Element(i64) → F 负 i64 归约 mod p（`F::from(i64)` 内部走 `From<i64>` 的符号→BigInt→fe 归约） batch 转域 `quantization/mod.rs:269-300`；`tensor/mod.rs:1883/1924-28` Tensor → Tensor 逐元素映射，形状不变

> **FPGA 定点**：Bn254Fr 模乘（Montgomery），加/减/乘 3 个蒙哥马利微操作。i64→Fr 最省做法：`|i64|` 用常数模的加减归约 + 符号位选择补 `p`。

1.2 张量 → MLE（`into_mle`）

微算子 源码位置 输入 → 输出 说明 **pad 到 2 的幂** `tensor/mod.rs:812-836` `pad_next_power_of_two` Tensor[..,s×c] → Tensor[..,s'×c', s'≥s 为2次幂] `shape.next_power_of_two()`；data 后补 `T::default()`(0)；逐维补到 2^k 使 numel=2^k。**元数据：记 unpadded_shape 平铺成 MLE** `tensor/mod.rs:400` `into_mle` / `data.into_mle()` Tensor data(Vec) → DensePolynomial(求值表) 行主序排布；下标 idx=$b_0+2b_1+\ldots+2^{k-1}b_{k-1}$ ↔ 求值 $\mathcal{M}(b_0..b_{k-1})$ EQ 用 MLE（段5） —— 到段5

> **FPGA 定点**：`pad` = 数据搬移 + 尾部补 0；`into_mle` = **无拷贝地重新解释**数据为布尔超立方体表（若数据在按行主序的连续内存，则就地，零运算）。**这是最简单的一类"算子"**：大多是 `repack`，不是 `arithmetic`。

段 2 · witness 批量承诺：`generate_lookup_witness_for_chunk` → `batch_commit`

> **入口链**（G2 §3.1）：`instantiate_witness_ctx_for_chunk`(iop/prover.rs:1114) → `generate_lookup_witness_for_chunk`(lookup/context.rs:224)。**每 block 最重的一步（≈57% 耗时）**。逐节点 `gen_lookup_witness` 产 witness MLE（如 GELU 的 input+output MLE），再 `forward_inners` 顺拓扑逐节点并行产，最后**一整块一次** `batch_commit`。

微算子	源码位置	输入 → 输出
逐节点 witness 生成	`lookup/context.rs:251-313` `gen_lookup_witness`	节点 step(i64 输入/输出) → witness MLE
查表计数 multiplicity	`lookup/context.rs:313`	各查表值出现次数 → `mult_i = 被查次数÷表出现次数`
批量切块	`poly_commit/context.rs:58-115`	MLE(2^k) → 按 2^20 切 k 个块（>1M 才切，上限128）
批量承诺 MSM	`poly_commit/context.rs:325` → `arkyper/mod.rs:471` `batch_poly_msm`	各块(2^20 标量) → 1 个 G1 点/块

> **MSM 微算子**（FPGA 重计算核心）：Pippenger MSM `C = Σ a_i·(g^{τ^i})`。
>
> - 拆标量到 radix-r 窗口位
> - **EC 点加/double**（G1 上仿射/雅可比加法定律，内含若干 Bn254Fr 模乘/模逆）
> - 桶累加 + 滑动求和归约
> - Montgomery/求逆每窗一次 **数据流**：Σ 个 2^20 块 → Σ 个 G1 承诺；块数 = `Σpolys ceil(2^poly_vars/2^20)` = `ΣQ/2^20`（G §3.2）。

> ⚠ **分清两类承诺（本段只讲 witness 批量承诺）**：prove 里共两种承诺，别混——

承诺类型	何时	内容	源码
模型/表承诺（一次性，setup 期）	`generate_contexts` 构建 ProverContext 时	①各层固有权重 MLE（LayerNorm/RMS 的 scale 等 model_polys）；②**lookup 查询表**的 `committed_columns::<F>()`（如 GELU 查表	`poly_commit/context.rs:170-214` + `lookup_context.iter()`

承诺类型	何时	内容	源码
		的输出列)。两者连同走一次 `batch_commit`，表用独占 `table_node_id` 标识	
witness 承诺（每 chunk）	prove_chunk 每块开头	逐节点的输入/中间输出 MLE + lookup multiplicity	`lookup/context.rs:224` + `poly_commit/context.rs:325`（本段主内容）

模型/表承诺**不进 chunk、不随 s 变**，全局共享，verifier 在 setup 就拿到对应承诺；本段段2 的 `batch_commit` 只覆盖 witness 层。**FPGA**：表承诺是少量固定 G1 点，可预置，无需计入 per-prove 的 MSM 清单。

段 3 · claims 生成：输出 claim + 边界 claim

入口链（G2 §3.2）：prove_chunk 先算 `output_claims_by_port`(:583-621) 与 `chunk_output_claims`(:627-666)，作为逆拓扑的种子。

微算子	源码位置	输入 → 输出
MLE 点求值 `evals(point)`	`poly_commit/prover.rs:172/201` → dp `poly/eq.rs`	MLE×点 → 求值(标量)
EQ 多项式构造/求值	`poly/eq.rs:18`	点 p → eq_p(x)=∏(pᵢxᵢ+(1-pᵢ)(1-xᵢ))
挑战点采样	`iop/prover.rs:158` 等 / dp transcript	消息→随机 F
边界 claim 合并	`iop/prover.rs:627-666`	外向边 tensor MLE + common_point → Claim

EQ 求值微算子：递归展开 `eq_p`（或按二元树自下而上折叠），每层 `(1-pᵢ)+(2pᵢ-1)·x` 线性组合 + 相邻元素乘。**这是 sumcheck 里最常见的基元**。成本 = O(2^k) 域乘/轮。

段 4 · 节点级 sumcheck：`op.prove`（einsum 等）

入口链（G2 §4）：主循环 `backward_iter`(:671) 每节点 `op.prove` → einsum `prove_internal`(layers/einsum/prove.rs:45) → `fix_axes` 把固定轴消去 → 虚拟多项式 → `IOPProverState::prove`(dp sumcheck/prover.rs:116)。

4.0 LLM 末 token 分支：只证一条 logits，不按 s 放大

LLM 推理只证明最后一个位置的输出，这是主链数据流必须先立起的形状前提（口径见 F §3/§4 末 token 注、model/llm.rs:537）。

- 全序列前向里，`final_proj` 产出 `[s, vocab]` 张量，但对**证明**而言**只取末 token（最后一行）的 logits**，其余 `s-1` 行的 logits **不进 sumcheck、不承诺**——它们对生成无影响、无需证明。
- 因此主链段3 的**输出 claim / 边界 claim 只回溯到最后一行**，`final_proj→argmax` 的 argmax 值（`proving_data`）单独参与证明（证"取到的是 argmax"），但不按 s 放大张量。
- **对 FPGA 的含意**：vocab(如 50257) 那一次 `[vocab]` 宽的 logits 计算 / logits 查表 sumcheck 是**常数规模的单行粒度**，不随 token 循环累加；序列长 s 只影响**前面层**的张量（仍按 s 走段1 的 `[·, s, ·]` 域化），不影响这一步的 instance 数。

其它（非 LLM、逐 token 输出都要证的）场景则每次输出都产一个 claim，本分支的起点是"trace 表 + 末 token 这个额外信息"。

4.1 einsum：数据切分/重排 + 虚拟多项式组装

微算子	源码位置	输入 → 输出
切分输出轴 (split_points)	einsum 相关	输出张量 → 各端口分配
fix_axes 三种轴策略	einsum/axis.rs`FixAxis::{Outer,Stacked,Contracted}`	全输入 MLE → lhs/rhs 分组 MLE

微算子	源码位置	输入 → 输出
Outer 轴：跨步点积	axis.rs:1184-1192	`skip(i).step_by(chunk).zip(beta).map(v·b).sum()`
Stacked 轴：扩 chunk	axis.rs:1200	扩 chunk 不扩收缩
Contracted 轴：求和轴消去	axis.rs:1215-1233	outer 固定后按收缩分组、补0到2^v → DensePolynomial
变量数	axis.rs num_vars	`num_vars = ceil_log2(contraction_size)`
虚拟多项式组装	einsum/prove.rs:184→`VirtualPolynomialsBuilder`	多 MLE 积 × batching_challenge

4.2 sumcheck 轮内微算子（dp `sumcheck_macro/src/lib.rs:62-357` + `sumcheck/prover.rs:356-383`）

核实口径：轮内 `build_uni_poly` 由 `sumcheck_code_gen!` 宏生成。对**每个产物**（d 个 MLE 的乘积），按布尔超立方体 `b` 步进 2 遍历；每个 `b` 覆盖 univariate 的 0/1 两侧。`v_j` = 该 MLE 的切片。

微算子	源码位置	输入 → 输出	每轮每产物数量	FPGA 判定
相邻差	宏`c0 = v[b+1]−v[b]`	两个相邻域元素 → 差	~len/2	sub
差倍增（域加，非移位）	宏`c_j = c_{j-1}+c_{j-1}`	前差 → 2^t·差	~len/2·⌈log2(deg)⌉	add（FPGA 上即自加，类似左移但走域加）
行系数组装（按 set-bit 选 c 链）	宏 rows	c 链 + v[b+1] → 各行系数	len/2·(deg−1)	add
行因子跨 MLE 相乘	宏`dot`	d 个 MLE 的同行因子 → row-product	len/2·(deg+1)·(d−1)	**mul（d−1 个/行）**
项系数缩放	prover.rs:375`uni_variate*scalar`	行值 × scalar	(deg+1)	mul
域乘幂/常数	`F::from(1<<num_vars_multiplicity)`	补乘 2^mult（可变少变量产物）	偶发	常数乘
外推补满次	`sumcheck/extrapolate.rs:159-173`	d 点值 → max_degree 点值	(maxdeg−d)·(deg+1)	**预计算常数点积：d+1 mul + d add**（运行期无 inverse）
累加进轮多项式	prover.rs`uni_polys += uni_variate`	各 term/产物 → $f_i(x)$	(deg+1)·#products	add
换维 fold（fix_low）	`dense.rs:477 fix_low` / prover.rs:333	MLE → 固定 r_i 减半	len/2 (sub+mul+add)	sub+mul+add，little-endian

FPGA 定点（sumcheck）：核心是**一行宽的 256-bit Montgomery mul 阵列** + sub/add 树。轮间**串行**（第 i 轮 challenge 后才进第 i+1），轮内所有 `b` 全并行。每轮先 fold（fix_low）再 build_uni_poly。外推在**预计算表建表时消掉 inverse**，运行期只是常数点积——**不要在板上放运行时求逆**。

FPGA 定点：sumcheck 是**串行轮链**（第 i 轮依赖第 i−1 轮的 MLE 折叠结果），每轮 `O(d²·2^{n-i})` 域乘。**它是最需要流水的：逐轮折叠 + 轮多项式累加**。这个算子是 G 里"重计算 + 逐轮"的难点。

段 5 · 合并 sumcheck + RLC（`commit_prover.prove`）

入口链（G2 §5）：`poly_commit/prover.rs:97 claims_batching_sumcheck`(:140) 把本块**所有 claim** 归约到单点，再 `linear_combination`(RLC) 塌缩成**一条**多项式，对单点一次打开。

微算子	源码位置	输入 → 输出
EQ 多项式 per claim	prover.rs:172/201`evals(poly.eq_point_per_chunk(point)).into_mle()`	每 claim → eq 多项式
batching 系数	poly_commit/mod.rs:46`batching_coefficients_for_chunks`	`C_j = eq(j, point 高位)` 按块
sumcheck 表达式组装	poly_commit/mod.rs:93`build_sumcheck_expression`	`挑战·(Σ_j C_j·P_j_chunk)·eq_p` 全项累加
合并 sumcheck	prover.rs:240`IOPProverState::prove`	上述虚拟多项式 → 单点
RLC 线性组合	dp`poly/dense.rs:934 linear_combination`	`rlc = Σ c_i·P_i`
挑战标量	transcript.challenge_scalar	消息 → 每 poly 一个 c

段 6 · KZG 打开：`HyperKZG::open`

> **入口链**（G2 §7）：`PCS::prove`(:497) 先求值再 `HyperKZG::open`(:329)。三阶段。

6.1 Phase1 折叠（2^m → 常数，轮间串行 / 轮内全并行）

微算子	源码位置	输入 → 输出	FPGA 判定
折叠递推	arkyper/mod.rs:349-356	`coeffs[j] = r_i·(p[2j+1]−p[2j]) + p[2j]`	每 j：sub→mul→add；每轮减半；**轮间串行**（polys[i+1] 依赖 polys[i]）
并行性	`cfg_iter!`	轮内所有 j 段互不依赖	**轮内全并行**（每轮一个 wide 扇出 sub/mul/add 级）

6.2 Phase2 承诺折叠中间多项式 + 采挑战

微算子	源码位置	输入 → 输出
批量承诺折叠多项式	arkyper/mod.rs:362`batch_poly_msm(pk.g1_powers, polys[1..])`	ℓ−1 个折叠 → ℓ−1 个 G1 承诺
挑战 r	transcript	消息→r
3 点构造	arkyper/mod.rs:373	`u=[r,−r,r²]`（neg、sqr、组装）

6.3 Phase3 3 点批量打开（kzg_open_batch :186）

微算子	源码位置	输入 → 输出	微操作
单变量 Horner 求值	dense.rs:765-773`eval_as_univariate`	折叠多项式 @u_j	每系数 1 mul+1 add；rpow 每步 1 mul；t×k 个并行 Horner
挑战幂	transcript`challenge_scalar_powers`	消息 → q^0..q^{len}	**串行 mul 链** q_powers[i]=q·q_powers[i-1]
再 RLC	dense.rs:934`linear_combination`	`b_poly[i]=Σ_j q_j·poly_j[i]`	per coeff：k mul + k add，并行
witness 多项式	mod.rs:171-183`compute_witness_polynomial`	逐点`h[i-1]=f[i]+h[i]·u`	**每系数串行 1 mul+1 add**（除以 x−u）
3 点 KZG open MSM	mod.rs:222`kzg_batch_open_no_rem` → `batch_poly_msm`	witness polys → (w_0,w_1,w_2)	MSM（附录D）

段 7 · 序列化 / 证明打包（postcard + ark_serialize）

> 收口后 `Proof`（`iop/mod.rs:102`）/`ChunkProof`(:67)/`TableProof`(:138)/`OpeningProof`(`poly_commit/mod.rs:155`) 经 **postcard** 序列化（提交 187d6c9b "use postcard for PPs & proof serialization"）。域元素用 `ark_serialize::CanonicalSerialize` 压缩

（Fr→32B，G1→2×32B，∞ 哨兵）。

微算子	源码位置	输入 → 输出	硬件
域→字节压缩打包	`ark_serialize::CanonicalSerialize` / `quantization/mod.rs SerializableField`	Fr → 32B LE；G1 → (x,y) 各32B	打包（bus 编解码）
proof 结构扁平化/拼接	postcard`serialize`	各段 proof → 连续字节	顺序打包 + 长度前缀
点/求值/轮多项式/commitment 逐项写入	`iop/mod.rs` 各 Proof 字段	段0~6 产物 → 证明文件	打包

数据流贯穿无遗漏的核对方式：沿 G2 入口：`chunked_prove_local`(段0) → `prove_chunk` 主循环 `:671`(段1~段3) → 每节点 `op.prove`(段4) → `commit_prover.prove`(段5) → `batch_commit`(段2/6.2) → `HyperKZG::open`(段6) → 序列化(段7)。每一段的输出正是下一段的输入，逐节点/逐 chunk 闭环。**本文档下一个版本可补**：把每个算子按 BN254/Fr/Fq/Blake3 的功能单元归并成一张"硬件单元 → 复用表层"图（已在附录给出雏形）。

附录 A · 数据切分 / 拼接 / 组装的微算子（以 SplitLayer / einsum / reshape 为例）

这类是**数据搬运型算子**（搬运 > 算术），FPGA 上是**地址生成 + 多路选择 + 少量 scalar 归约**，不是模加减乘。用户点名的"i64 转 fr、数据的切分、拼接、组装"集中在此。

A.1 Tensor 切分（SplitLayer, `layers/split.rs`）

微算子	源码位置	输入 → 输出	FPGA 判定
pad 到 chunk 形状	`split.rs:273 .pad(new_shape, 0)`	tensor → 补0到可切形状	补0 + 元数据
按维 0 切块	`split.rs:274 .chunk(num_chunks, 0)`	tensor → N 个 chunk	**地址切分/分段**（连续内存行切）
丢弃补齐 chunk/还原	`split.rs:275-283 truncate+reduce_to_shape`	切出的 chunk → 最后一块去掉 padding	搬运
claims 数组切片	`split.rs:223 split_output_claims`	Vec → 按 chunk 切	索引

A.2 Split 的 claim 合并（数据回图：把分块的 claim 拼回输入 claim）

微算子	源码位置	输入 → 输出	公式
补点坐标求值缩放	`split.rs:373-377`	padding_point → scalar	`eval · ∏(1−padding_coord)`
EQ/challenge 组合	`split.rs:198 evals(&challenges)`	挑战 → β 表	`eq` 求值
跨 chunk RLC 合并	`split.rs:200-203`	β + chunk_evals → combined	`acc + eval·β`（fold）
点坐标拼接	`split.rs:206 .chain(challenges)`	point + challenges → 新 point	concat
不一致 claim 时 sumcheck	`split.rs:399-433`	输出 MLE +`evals(padded_point).into_mle()` EQ → sumcheck	走段4 sumcheck

A.3 einsum 的轴切分/组装（`layers/einsum/axis.rs` + `prove.rs`）

微算子	源码位置	输入 → 输出
claim 点按输出形状切分	`prove.rs:138-142` `shape.split_point(claim.point())`	多点 → 各端口 segment
固定轴/求和轴分组	`axis.rs` `fix_axes`	全输入 MLE → lhs/rhs 分组
收缩轴拆解、按收缩分组、补0到 2^v	`axis.rs:1215-1233`	outer 固定后 evaluation → DensePolynomial
跨步点积（Outer 轴）	`axis.rs:1184-1192`	`skip(i).step_by(chunk).zip(beta).map(v·b).sum()`
stacking 系数折叠	`prove.rs:242` `stacking_coeffs::fold(acc + c·e)`	coeff + eval → scalar
重插回完整点（interleave 组装）	`prove.rs:152/337-361` `reconstruct_full_point`	固定点段 + sumcheck 点段 →**按输入形状交错回完整坐标**
EQ 多项式重建	`prove.rs:264-273` `evals(eq_point) → DensePolynomial`	点 → EQ MLE

> **FPGA 定点**：`reconstruct_full_point` / `split_point` 是**坐标级互插/拆分**（按维度对数 bit 数切 or 拼），是乘法累加友好但需要**索引调度器**的搬运微操作。`acc+c·e` 是 1 个 mul+add（RLC 基元）。

附录 B1 · Lookup 算子（LogUp 查表 + 链接 sumcheck）— GELU/softmax/LN/rmsnorm/logits 共用

> **入口链**：softmax/activation/LN 等层的 `prove` → `prove_lookup_op`(`lookup/operation/generic_prove.rs:66`)。这里是**查表型层（GELU、softmax、norm、logits）的全部证明**：先把每个节点的 i64 输入/输出域化成 witness MLE（段1/段2已建模），然后 LogUp 证明查表，再一个链接 sumcheck 把 table 输出和输入对齐。**weight 张量（norm 用）还要逐元素乘 lookup 输出**。

微算子	源码位置	输入 → 输出	说明
LogUp batch prove	`new_batch_multiple_sizes_prove`（logup_gkr/prover.rs:91）	各 chunk 查表 MLE + multiplicity → logup proof + 点	LogUp 分数和恒等，多层 GKR 逐层 sumcheck
LogUp 各层累加表达式	prover.rs:224-250	每层分数和项 →∑ `expr·Eq_poly`	sumcheck 表达式拼接
power-of-two 系数缩放	prover.rs:222`F::from(1<<(total_layers-size))`	不同尺寸实例 claim → 对齐	常数乘
低/高半合并（batching_challenge）	prover.rs:257`compute_current_claim`	`rk·f(r..,1)+(1−rk)·f(r..,0)`	RLC 基元
EQ 多项式（当前层点）	prover.rs:247-249`evals(&sumcheck_point[..]).into_mle()`	点段 → EQ 2^k 表	附录D.2
把 lookup MLE 按 chunk 分组地切片	`generic_prove.rs:108` `split_at(number_of_chunks*...)`	全 layer MLE → lookup_polys + others	数据切分
normaliser MLE 扩维/复制广播	`generic_prove.rs:183-192` `evals.flat_map(repeat_n(v,padded_final_dim)).into_mle()`	1 个值 → 复制成 padded_final_dim 个	**数据复制/广播**
weight 张量重复/拼接	`generic_prove.rs:209-219` `vec![weight_field;second_last_dim].concat().into_mle()`	weight MLE → 复制 second_last_dim 份并接	**数据复制+拼接**
链接 sumcheck	`generic_prove.rs:222` `prove_linking_sumcheck`	lookup输出MLE + table输出 = 源输出（含 weight 逐元素乘）	段4 sumcheck

微算子	源码位置	输入 → 输出	说明
dim_lt 多项式求值	`evaluate_dim_lt_poly`	点 + dim → 1/lt 指示	级联比较
MLE 求值 + 除以 lt 值（field 除）	`generic_prove.rs:247-257` `m.evaluate(shift_point) * lt_eval`	MLE@点 → scalar	mul
inverse + 乘（shift 归真）	`generic_prove.rs:305-316` `eval * inverse(lt_eval)`	scalar → scalar/lt	**field inverse**
构造各端口 claim	`generic_prove.rs:260-290`	多个 eval → Claim	组装

附录 B2 · Embedding / Token-index 的散点组装（`layers/transformer/embeddings.rs:366-465`）

这是"从 token 索引到嵌入矩阵行"的证明，是**数据组装的典型**：输入是 `x[i]`（i64 整数 token 索引），要证"输出 = 嵌入矩阵按索引取行"。prover 用**稀疏 one-hot 散点累加**把索引映射归约成一个可域化向量，再对 `reduced_one_hot · 嵌入矩阵` 做一次 sumcheck。

微算子	源码位置	输入 → 输出	FPGA 判定
claim 点按行列拆段	`embeddings.rs:389` `split_output_point`	last_claim.point → (row_point,column_point)	point 切分
EQ 位置 β 表	`embeddings.rs:397` `beta_vec = evals(row_point)`	row_point → 每位置 β	附录D.2
★ sparse scatter-add（索引散点累加）	`embeddings.rs:401-410` `reduced_one_hot[x[i]] += beta_vec[i]`	索引序列 + β → vocab 长累加向量	**按索引写累加器（gather/scatter 内存）**
嵌入矩阵 pad/域化	`embeddings.rs:414` `pad_next_power_of_two`	权重(i64) → Fr 矩阵	段1
MLE 固定列变量（fold）	`embeddings.rs:432` `fix_low_variables_in_place_parallel(column_point)`	嵌入 MLE → 固定列点后减半	段6.1 同款 fold
sumcheck（2 MLE 乘积）	`embeddings.rs:443-444`	`input_expr * embedding_mat_expr` → sumcheck	段4.2
构造 one-hot/嵌入 claim	`embeddings.rs:453-461`	轮挑战 + 求值 → 2 Claim	组装

FPGA 定点：`reduced_one_hot[x[i]] += β` 是典型的**索引寻址累加**（读-加-写回一个 vocab-size 的 Fr 缓冲），`x[i]` 是 i64 索引 → 地址；`select/gather`（`weights.select(0, input.int())`，行 204）在推理里做、prove 里用 one-hot matmul 替代。索引类型 i64→地址 → 累加，是 FPGA 上"几乎零算术、纯数据通路"的算子，但**内存随机写**需要 bank 冲突处理。

附录 B3 · 卷积 / FFT 批量证明的算子（`layers/convolution/mod.rs:698+`）+ 位置编码

卷积证明走 **FFT 域批量 sumcheck**（`BatchFFTProof`）：`Y[i]=iFFT(Σ_j FFT(X[j])∘FFT(W[i][j]))`。注意它仍在 `Fr` 上用 sumcheck 证并矢，**不是 Fr 上的 NTT**——"FFT"是把这个关系写成一个 GKR/多重 sumcheck 表达。

微算子	源码位置	输入 → 输出
filter→FFT 张量重排	`mod.rs:706` `as_fft_tensor`	filter → FFT 布局, pre_fft_shape
挑战点拆 (r1,r2)	`mod.rs:711-719`	r → r1(行) + r2(列) 段
EQ β	`mod.rs:724` `beta = evals(&r2)`	r2 → β 表

微算子	源码位置	输入 → 输出
★ RLC 加权散点累加 filter	`mod.rs:736-744` `w1_reduced[k] += beta[ij]·tensor[idx].to_field()`	(i,j,k) 重排 + 域化 + mul+add 累加
φ-G GKR 折叠	`mod.rs:725` `phi_g_init`	r1 + F::ONE → w_red + f_middle
系数重排 (index_wf)	`mod.rs:747-748` `index_wf`	w1_reduced → 按 FFT 形状重排
sumcheck（2 MLE 乘积）	`mod.rs:756-762`	f_m·f_red → sumcheck
矩阵求值委托	`mod.rs:768` `delegate_matrix_evaluation`	f_middle @ r1 → 打开
位置编码	`positional/mod.rs:33-231` `PositionalCtx::{Absolute,Rope}`	见下（两种 variant 证明路径不同）

位置编码的证明路径（补齐段1~段4 数据流的分支）：`PositionalCtx` 有两种 variant，别当成单一 sin/cos 表——

Variant	是什么	怎么证明	典型模型
Absolute	**学习到的绝对位置 embedding**（矩阵行 = 每个位置一个向量）	**走附录 B2 的 Embeddings 证明**：position-index → one-hot 散点累加 → sumcheck，与 token embedding 同一条路径（GPT-2 的 positional 权重通常 tied 到 token embedding 矩阵）	GPT-2
Rope	RoPE 旋转位置编码，`cos/sin` 预计算矩阵	cos/sin 矩阵是**模型级权重 MLE**（`base_frequency_id` 提交）；旋转 = 对 Q/K 张量做逐元素 `cos·x + sin·x'` 线性组合，作为对 query/key 的**系数作用**参与 einsum/sumcheck，本身单独成节点证明	Llama/Qwen/Gemma

对 FPGA 含意：Absolute 就是 Appendix B2 一次额外的散点组装（position 索引固定 0..s，比 token vocab 小得多）；RoPE 则是矩阵-向量系数作用，走段4 einsum 同款 sumcheck，占一条小的线性张量。两者都不放大成独立巨型张量，但**都必须在 prove 顺序里作为 Embeddings/系数层被消费**（不是 trace 里随手可丢的旁支）。

附录 B4 · 注意力掩码 / 相对位置（`layers/transformer/attention_mask/prove.rs`）

微算子	源码位置	输入 → 输出	说明
固定行/列变量的 MLE 归约（fix_mle_rec）	`attention_mask/prove.rs:13-56`	EQ`fix_mle_rec(row_point)` / `(column_point)`	递归固定变量生成部分求值 EVAL 表（同附录D.2 EQ，但带`update_acc` 累加）
EQ 多项式装箱	`prove.rs:79-80` `DensePolynomial::new(eq_evals)`	部分求值 → EQ MLE	段5 用

附录 C · 域求逆 / 除法（BN254Fr）

微算子	源码位置	输入 → 输出	说明
域减	所有`-`	a,b → a-b	蒙哥马利域减（+p 归约）
域乘	所有`*`	a,b → a·b mod p	**BN254 模乘核心**（蒙哥马利，254bit）
域加/累加	`fold(ZERO, acc+..)`	序列 → Σ	多周期累加器
求逆	`quantization/mod.rs:309` / `generic_prove.rs:309` `.inverse()`	a → a^{p-2} mod p	**BN254 求逆 = 254-254bit 幂（模平方+模乘的链）/ 或 CIOS 扩展欧几里得**；对 FPGA 是"重除法单元"

微算子	源码位置	输入 → 输出	说明
除以标量	`eval * inverse_lt_eval`	a, b → a/b	分解为 inverse+mul

附录 D · 底层密码微算子（Bn254 域 + MSM + EQ + transcript）— dp-crypto 精确定点

> 核实：dp-crypto 全程**无 NTT/FFT/小域/扩张域**——全流程就是 **Fr 标量域算术 + G1 点算术 + Blake3 hash**。Fr 与 Fq（MSM 的曲线坐标域）都是 254bit 蒙哥马利、4×u64 肢。**FPGA 勿建 NTT 单元**。

D.1 域算术（ark_ff `Fp256<MontBackend>`）

微算子	用途	硬件
域加（+p 条件减）	eq/累加/fold	256b 加 + 条件减 p
域减	差/折叠	add of neg
域乘（**蒙哥马利 CIOS**）	**吞吐瓶颈**	256×256 Mont 乘（4-肢），sumcheck/fold/RLC/MSM 标量数码共用
域取负	-r	256b 减 p
求和累加器（加法树）	.sum()	流水加法树
求逆（Fermat p-2 幂）	仅**verifier/预计算表**	可放慢/小面积单元，**prover 热路径无需**

D.2 EQ 多项式（`poly/eq.rs:54-141`，`virtual_poly.rs:346`）

微算子	每元素	并行度
递推展开`evals[i]=r·scalar`，`evals[i/2]-=..`	1 mul + 1 sub	**每层全并行，仅 n 个串行层**
单点`eq_eval`: `res*(xi·yi))+res*(..)`	1 mul + 若干 add/sub + 1 mul 累加	—

D.3 MSM Pippenger（`arkyper/msm.rs`，最终落到 ark-ec/ec-gpu-gen）

微算子	说明	硬件
标量→radix-c 窗口数码	Pippenger 分窗	位抽取
每窗口桶累加	scalar 数码 → bucket，**G1 点 double+add**	**Fq 基域点运算 + Fr 标量 mul**
跨窗口组合	每窗 c 次 double + add	同上
曲线坐标	$y^2=x^3+3$ over Fq	需要**Fq 域（254b）蒙哥马利**（与 Fr 相似）

D.4 Fiat-Shamir transcript（`arkyper/transcript/blake3.rs`）

微算子	输入 → 输出	硬件
吸收 append_bytes	序列化字节 → blake3(state)	blake3 压缩（1024-bit 块，7 round，state 512b）+ XOF
挤压 challenge	`finalize_xof().fill(16B)` → `F::from_le_bytes_mod_order`	**128bit little-endian → 256b 归约 mod p（rejection-free，因 2^128<p）**
域→字节	Fr → 32B 压缩; G1 → 2×32B(x,y)+∞哨兵	打包
挑战幂	q^i 逐次 mul	串行 mul 链

> **FPGA 定点核心结论**：整条 prove = **一块 254-bit 蒙哥马利域单元**（Fr/Fq 复用）+ G1 点运算（MSM/打开）+ Blake3 + 数据搬运（切分/复制/拼接）+ 逐节点/合并 sumcheck。**串行瓶颈点**（调度器要暴露流水）：sumcheck 轮间、折叠 **ell** 轮、witness Horner、挑战幂链。**并行富矿**：sumcheck 轮内 b、EQ 层内、fold 轮内 j、MSM 桶。

关键技术事实（FPGA 边界）

- **重计算算子 = 3 类**：批量 MSM（段2/段6）、sumcheck + EQ/fold（段4/段5/段6.1）。
- **必配的配套算子不能缺**：i64→Fr（段1）、转 MLE（段1）、EQ 生成（段3/5）、RLC（段5/6.3）、**挑战采样/transcript（Blake3，全程）**。这些留主机会导致逐轮交互爆炸（G §5.4）。
- **目标 FPGA 形态 = 端到端 prove 内核**：主机只喂 `trace + 权重`，FPGA 内部跑完「切 chunk→域化→批量承诺→逆拓扑 sumcheck→合并+RLC→KZG 打开」，只在**首尾/边界**交互。挑战采样必须贴近/下放 FPGA。
